JavaScript 2

Ambient intelligence course (A.Y. 2025/2026)
Topic taught by Prof. Davide Loconte

Gabriele Colapinto

April 27, 2026

License

Notes from Ambient intelligence (A.Y. 2025/2026) by Gabriele Colapinto. Course taught by professors Michele Ruta, Floriano Scioscia, Arnaldo Tomasino, Davide Loconte — Licensed under CC BY-NC 4.0.
https://creativecommons.org/licenses/by-nc/4.0/

© 2025 Gabriele Colapinto
Released under the Creative Commons Attribution–NonCommercial 4.0 International License

Contents

1 The Core of JavaScript Objects: Prototypes **7**
1.1 prototype vs. __proto__ . . . 7
1.2 Class-based vs Prototype-based Inheritance . . . 8
1.2.1 Class-based Inheritance . . . 8
1.2.2 Prototype-based Inheritance . . . 8
1.2.3 Dynamic vs Static Nature . . . 9
1.2.4 Object Structure and Evolution . . . 9
1.2.5 Memory Usage . . . 9
1.2.6 Performance Considerations . . . 10
1.2.7 Flexibility vs Safety . . . 10
1.2.8 Why JavaScript Uses Prototype-based Inheritance . . . 10
1.2.9 Summary . . . 11

2 The Mechanism of Inheritance: The Prototype Chain **13**
2.1 Property Lookup Process . . . 13
2.2 The Mystery of Primitives . . . 14

3 Practical Implementation: Creating and Linking Objects **15**
3.1 Delegation via Object.create() . . . 15
3.2 Copying via Object.assign() . . . 15
3.3 Comparative Analysis: create vs. assign . . . 16
3.4 Interaction between the this keyword and the prototype chain . . . 16

4 The Compiler's First Pass: Hoisting **19**
4.1 The Temporal Dead Zone (TDZ) . . . 19
4.2 Examples: comparing var, let, and const . . . 19
4.3 Why TDZ Exists . . . 20
4.4 Two-Pass Execution Model . . . 20

5 Modern Abstraction: ES6 Classes as Syntactic Sugar **23**
5.1 Deconstructing "Syntactic Sugar" . . . 23

6 The Core Execution Engine: How JavaScript Runs Code **25**
6.1 The JavaScript Runtime Environment . . . 25
6.2 Memory Management: The Heap and The Call Stack . . . 25
6.3 The Execution Context (EC) and Hoisting . . . 26
6.3.1 Global Execution Context . . . 26
6.3.2 Two Phases of Execution Context Creation . . . 26
6.3.3 Internal Structure of an Execution Context . . . 27
6.3.4 Example: Memory creation phase and execution phase of a simple print function . . . 27
6.3.5 Visual Model . . . 28
6.3.6 Call Stack Behavior . . . 28

6.3.7 Key Observations . . . 29

7 The Concurrency Model: Event Loop and Queues 31
7.1 The Problem with a Single Thread . . . 31
7.2 The Components of Asynchronous Execution . . . 31
7.3 The Order of Operations . . . 32
7.4 Example of synchronous code execution . . . 33
7.5 Example of asynchronous code execution . . . 34

8 Managing Scope and State: Closures 37
8.1 Lexical Scoping . . . 37
8.2 Defining a Closure . . . 38
8.3 Practical Application of Closures . . . 38

9 Asynchronous Operations with Promises 39
9.1 The Promise Object and its States . . . 39
9.2 Creating and Consuming Promises . . . 39
9.2.1 Creating a Promise . . . 39
9.2.2 Consuming a Promise . . . 40
9.3 Promise Chaining and async/await . . . 40
9.4 Advanced Promise Patterns . . . 41
9.4.1 Concurrent Operations . . . 42
9.4.2 Advanced Error Handling . . . 42
9.5 Example: Call Stack, Microtasks, and Macrotasks . . . 42
9.5.1 Expected Output . . . 43
9.5.2 Step-by-Step Execution . . . 43
9.5.3 Key Observations . . . 43

10 Structuring Code: Object Creation Patterns 45
10.1 Foundations of Design Patterns in JavaScript . . . 45
10.2 The Object Decorator Pattern . . . 45
10.2.1 Example of decorator pattern usage . . . 46
10.3 Functional Classes . . . 47
10.3.1 Example of functional class usage . . . 48
10.4 Functional shared pattern . . . 48
10.4.1 Example of functional shared pattern usage . . . 49
10.5 Functional Shared Pattern with Closure (Hybrid Pattern) . . . 50
10.5.1 Example of hybrid pattern usage . . . 50
10.6 Prototypal Classes . . . 51
10.6.1 Example of prototypal class usage . . . 52
10.6.2 Example of prototypal class usage with official conventions . . . 52
10.7 The Pseudoclassical Pattern . . . 53
10.7.1 Example of pseudoclassical pattern usage . . . 54
10.8 Superclasses and Subclasses . . . 55
10.8.1 Example of superclasses and subclasses usage . . . 55
10.9 Pseudoclassical Subclasses . . . 57
10.9.1 Example of pseudoclassical subclasses usage . . . 57
10.10 The ES6 Class-based Pattern . . . 62
10.10.1 Example of Class-Based Pattern usage . . . 63

10.11 Synthesis and Pattern Comparison Summary 64
10.11.1 Architectural Comparison Matrix 64
10.11.2 Decision Matrix for Architects 64
10.11.3 Conceptual Unification 64
10.11.4 Final Remarks 65

1 The Core of JavaScript Objects: Prototypes

To truly understand inheritance in JavaScript, one must first grasp the concept that lies at its very foundation: the prototype. Unlike classical object-oriented languages that rely on classes, JavaScript is built upon the idea of prototypal inheritance. Every object in JavaScript has a direct link to another object, its prototype, from which it can inherit properties and methods. This simple yet powerful mechanism is the key to understanding how objects relate to and share behavior with one another.

A JavaScript prototype is an object from which other objects inherit properties. When you create a function or an object, JavaScript automatically adds a hidden property that points to this prototype object, establishing a link that forms the basis of the inheritance model. Understanding the distinction between the properties used to manage this system—prototype and ___proto___—is crucial.

1.1 prototype vs. ___proto___

While their names are similar, these two properties serve different functions in the prototypal inheritance system.

Property	Exists On	Purpose
prototype	Functions and Classes	An object that serves as a **blueprint** for all instances created from the function or class. Methods and properties added here are shared among all instances.
___proto___	Every object instance	An internal property that points to the **actual prototype object** the instance inherits from. It is the link in the prototype chain.

This relationship becomes clear when we look at a constructor function and an instance created from it. Consider the following Person function:

```
function Person(name) {
  this.name = name;
}

Person.prototype.sayHi = function() {
  console.log("Hi, I'm " + this.name);
};

const p1 = new Person("Shejan");
```

When the p1 object is created using the new keyword, its internal __proto__ property is set to point directly to the Person function's prototype object. This establishes the inheritance link.

We can verify this connection directly:

```
// This assertion proves the link between the instance and the constructor's prototype.
console.log(p1.__proto__ === Person.prototype); // true
```

1.2 Class-based vs Prototype-based Inheritance

Object-oriented programming can be implemented using different paradigms. Two of the most relevant are:

- **Class-based inheritance** (e.g., Java, C++)
- **Prototype-based inheritance** (e.g., JavaScript)

Although both paradigms aim to support code reuse and abstraction, they differ significantly in design philosophy, flexibility, and behavior.

1.2.1 Class-based Inheritance

In class-based languages, objects are instances of classes, and classes define the structure and behavior of objects.

```
class Animal {
    speak() {
        console.log("Generic sound");
    }
}

class Dog extends Animal {
    speak() {
        console.log("Bark");
    }
}
```

Key characteristics:

- Classes act as **blueprints**
- Objects are created as **instances of classes**
- Inheritance is defined **statically** using class hierarchies
- The structure of objects is typically **fixed at compile-time**

1.2.2 Prototype-based Inheritance

In JavaScript, objects inherit directly from other objects through a mechanism called the **prototype chain**.

```
let animal = {
    speak() {
        console.log("Generic sound");
    }
};

let dog = Object.create(animal);
dog.speak(); // "Generic sound"
```

Key characteristics:

- Objects inherit from other **objects**, not classes
- Inheritance is **dynamic**
- Objects can be modified at runtime
- Properties are resolved through the **prototype chain**

1.2.3 Dynamic vs Static Nature

Prototype-based inheritance is more flexible than class-based inheritance.

Prototype-based (dynamic):

```
animal.walk = function() {
    console.log("Walking");
};

dog.walk(); // Works even if added later
```

Class-based (static):

In languages like Java, modifying a class at runtime is not possible in standard scenarios. The structure is fixed after compilation.

1.2.4 Object Structure and Evolution

- In class-based systems, objects must conform to their class definition.
- In prototype-based systems, objects can evolve dynamically by adding or removing properties.

1.2.5 Memory Usage

- **Class-based:** Methods are typically shared via the class, so memory usage is efficient.
- **Prototype-based:** Methods defined on the prototype are also shared across objects.

Important: Both paradigms can be memory-efficient if used correctly. However, in JavaScript:

- defining methods inside constructors creates duplicates
- defining methods on the prototype shares them

```
function Person(name) {
    this.name = name;

    // BAD: new copy for each object
    this.sayHello = function() {
        console.log("Hello");
    };
}
```

1.2.6 Performance Considerations

- Class-based languages often benefit from compile-time optimizations.
- JavaScript engines (e.g., V8) optimize prototype access using techniques such as inline caching (caching the prototype object address instead of traversing the prototype chain every time).

Observation:

- Prototype lookup introduces a small overhead
- Modern engines minimize this cost effectively

1.2.7 Flexibility vs Safety

- **Prototype-based:**
  - Highly flexible
  - Easier to modify and extend at runtime
  - More prone to errors and unpredictable behavior
- **Class-based:**
  - More rigid and structured
  - Easier to reason about large systems
  - Stronger guarantees (especially with static typing)

1.2.8 Why JavaScript Uses Prototype-based Inheritance

JavaScript was designed to be:

- simple and accessible
- flexible for dynamic web environments

Prototype-based inheritance supports:

- rapid development
- dynamic object manipulation
- flexible design patterns

Note: Modern JavaScript introduces the class syntax, but it is syntactic sugar over the prototype system.

1.2.9 Summary

- Class-based inheritance emphasizes **structure, safety, and predictability**
- Prototype-based inheritance emphasizes **flexibility and dynamism**

Feature	**Class-based**	**Prototype-based**
Inheritance	Static	Dynamic
Structure	Fixed	Flexible
Typing	Often static	Dynamic
Modification	Compile-time	Runtime
Memory	Shared methods	Shared via prototype
Performance	Compile-time optimized	Runtime optimized (JIT)

With this foundational concept of individual prototypes established, we can now explore how these links are chained together to create a powerful and efficient inheritance mechanism.

2 The Mechanism of Inheritance: The Prototype Chain

The strategic importance of the prototype chain cannot be overstated; it is the very mechanism through which JavaScript implements inheritance. This chain allows objects to access properties and methods from other objects, creating a hierarchy of shared functionality. Without it, every object would exist in isolation, with no way to inherit behavior.

The **prototype chain** is the series of links between objects, connected via their __proto__ properties. When you try to access a property on an object, JavaScript doesn't just look at the object itself; it traverses this chain until it finds what it's looking for.

2.1 Property Lookup Process

The property lookup process follows a clear and predictable sequence:

1. **Check the object itself:** JavaScript first searches for the property directly on the base object.
2. **Follow the __proto__ link:** If the property is not found, JavaScript follows the object's __proto__ property to its prototype and searches there.
3. **Continue up the chain:** This process continues recursively, moving from one prototype to the next up the chain until the property is found. If the end of the chain is reached (__proto__ is null), the property is considered undefined.

Consider this multi-level object structure, where info3 inherits from info2, which in turn inherits from info1:

```
const info1 = { fName1: "Shejan", lName1: "Mahamud" };
const info2 = { fName2: "Boltu", lName2: "Mia", __proto__: info1 };
const info3 = { fName3: "Habu", lName3: "Mia", __proto__: info2 };

// Accessing a property far up the chain
console.log(info3.fName1); // "Shejan"
```

Let us expose this example to highlight the difference between the actions of the JS engine and the user perspective.

The example is illustrated in figure 2.1. When info3.fName1 is requested, JavaScript checks info3 (not found), then its prototype info2 (not found), and finally info2's prototype info1, where fName1 is successfully located. At this point we have to distinguish the actions of the engine from what happens according to the user perspective. When the engine finds fName1, it can simply return it to the user without passing this value back in the chain so that info3 can return it. To the eyes of the user, instead, it seems like info3 has the requested property so the user

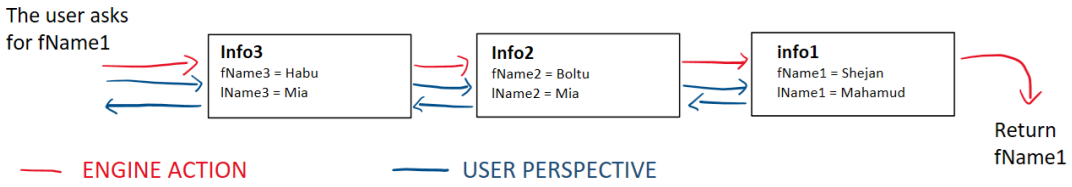

Figure 2.1: Prototype chain

thinks that they are communicating only with info3 who has all the information contained in the prototype chain.

2.2 The Mystery of Primitives

This lookup mechanism also explains a common JavaScript "mystery": how primitive types like strings and numbers can have methods. This is possible through a process called **"auto-boxing"** or **"wrapper objects"**. When you try to access a method on a primitive, like name.toLowerCase(), JavaScript performs these steps behind the scenes:

1. It temporarily wraps the primitive value in a corresponding object (e.g., new String("value")).
2. This temporary object's ___proto___ points to the appropriate prototype (String.prototype, Number.prototype, etc.).
3. The method is found on the prototype and executed.
4. The temporary wrapper object is discarded, and the value returns to being a simple primitive.

This elegant system is why it's often said that "everything in JavaScript behaves like an object," even when that's not technically true. Now that we understand the theory of the prototype chain, let's turn to the practical methods used to create and manage these inheritance relationships.

3 Practical Implementation: Creating and Linking Objects

This section serves as a practical guide to the two primary built-in methods for establishing relationships between objects: Object.create() and Object.assign(). These methods provide developers with direct control over how objects are linked or copied. Choosing the correct method is essential and depends entirely on whether the goal is to establish a dynamic, ongoing delegation or to create a static, one-time copy of an object's properties.

3.1 Delegation via Object.create()

The Object.create() method's core function is to create a new, empty object and set its internal ___proto___ reference to an existing object passed as an argument.

This method establishes a "live" reference, a form of **delegation**, not a copy. The new object is a lightweight shell that delegates property lookups to its prototype. Any changes made to the prototype object will be immediately reflected in all objects that inherit from it.

This behavior is illustrated in the following example. When A.a is modified, the change is visible through B because B is merely pointing to A as its prototype.

```
var A = {a: 10};

// B is created with A as its prototype
var B = Object.create(A);

console.log("B.a is: " + B.a); // 10

// Modifying the prototype object A
A.a = 20;

// The change is reflected in B through the prototype chain
console.log("B.a after changing A.a is: " + B.a); // 20
```

The key advantage of this delegation-based approach is **memory efficiency**. If the prototype is a "heavyweight" object with many properties, Object.create() allows you to create new objects that inherit its functionality without duplicating all of that data in memory. The new object is just a lightweight reference.

3.2 Copying via Object.assign()

In contrast, the Object.assign() method's purpose is to copy all enumerable own properties from one or more source objects directly onto a target object.

This operation creates a **shallow copy** of the source's properties, effectively breaking the live prototype link. The target object receives its own copies of the source's properties at the time of the assignment. Subsequent changes to the source object will *not* affect the target.

The following example demonstrates this detached relationship. After B is created as a copy of A, changing A.a has no effect on B.a.

```
var A = {a: 20};

// B is created by copying properties from A into a new empty object
var B = Object.assign({}, A);

console.log("B.a is: " + B.a); // 20

// Modifying the original object A
A.a = 15;

// B remains unchanged because it is a separate copy
console.log("A.a after change is: " + A.a); // 15
console.log("B.a is still: " + B.a); // 20
```

Warning for Deep Cloning It is important to note that Object.assign() performs a *shallow* copy. If a source property's value is a reference to another object, only the reference is copied, not the object itself. Both the original and the copy will point to the same nested object.

3.3 Comparative Analysis: create vs. assign

The choice between these two methods depends on the desired relationship between the objects.

Feature	Object.create()	Object.assign()
Core Purpose	Creates a new object using an existing object as its prototype.	Copies enumerable own properties from source(s) to a target object.
Relationship Created	**Ongoing lookup-time delegation**. Changes to the prototype are reflected.	**One-time property copying**. Creates a detached snapshot of properties.
Memory Impact	High efficiency; creates a lightweight reference to the prototype.	Lower efficiency; duplicates properties, consuming more memory for large objects.
Primary Use Case	Implementing prototypal inheritance and sharing functionality.	Merging multiple objects or creating a static snapshot of an object.

Understanding how to create and link objects sets the stage for examining the more nuanced behaviors that can arise within these prototypal structures, such as how JavaScript determines the execution context.

3.4 Interaction between the this keyword and the prototype chain

The value of the this keyword is a frequent point of confusion, but its behavior follows a simple rule: its value is determined by the object that *calls* the function at execution time, not by where

the function is defined. It refers to the context of the call.

This principle is clearly illustrated in the following example, where the bar object inherits the show method from its prototype, foo.

```
var foo = {
    message: "Hello",
    show() {
        console.log(this.message);
    }
};

var bar = {
    message: "World",
    __proto__: foo
};

bar.show(); // Prints "World"
```

Although the show method is defined within the foo object, it is called on the bar object (bar.show()). Therefore, the context for this inside the function becomes bar. When this.message is accessed, it resolves to the message property on bar, which is "World". The message property on foo is effectively **shadowed**, demonstrating that the call-site, not the definition-site, determines the context.

4 The Compiler's First Pass: Hoisting

Hoisting is JavaScript's behavior of processing variable and function declarations before any part of the code is executed. During an initial pass, the JavaScript interpreter allocates memory for all declarations, effectively moving them to the top of their scope. However, the way it treats functions and variables differs significantly.

- **Functions:** The *entire function declaration* (both the name and the body) is hoisted. This allows a function to be called before it physically appears in the code.
- **Variables (**var**):** Only the *declaration* (var varA;) is hoisted, not the assignment (= "Hello"). This means the variable exists from the top of its scope, but its value will be undefined until the line where it is assigned is reached during execution.
- **Variables (**let **and** const**):** These declarations are also hoisted, but they behave differently. Instead of being initialized with undefined, they are placed in a special state called the **Temporal Dead Zone (TDZ)** from the start of the scope until their declaration is evaluated.

4.1 The Temporal Dead Zone (TDZ)

The **Temporal Dead Zone** is the time between entering a scope and the actual declaration of a variable declared with let or const. During this period, the variable exists but cannot be accessed.

Attempting to use it results in a ReferenceError.

```
// Example: Accessing a let variable in TDZ
function test() {
    console.log(a);
    let a = 10;
}

test(); // ReferenceError: Cannot access 'a' before initialization
```

This behavior contrasts with var, which would return undefined instead of throwing an error.

4.2 Examples: comparing var, let, and const

```
// Example 1: Variable not declared at all
function printVar() {
    console.log(varA);
}
printVar(); // ReferenceError: varA is not defined
```

```
// Example 2: var hoisting
function printVar() {
    console.log(varA); // varA exists but is undefined
    var varA = "Hello";
}
printVar(); // undefined

// Example 3: let in TDZ
function printLet() {
    console.log(varB);
    let varB = "Hello";
}
printLet(); // ReferenceError

// Example 4: const in TDZ
function printConst() {
    console.log(varC);
    const varC = "Hello";
}
printConst(); // ReferenceError

// Example 5: Correct usage of let
function test() {
    let x = 5;
    console.log(x); // 5
}
test();

// Example 6: const must be initialized
const value = 10; // OK
const anotherValue; // SyntaxError
```

4.3 Why TDZ Exists

The Temporal Dead Zone was introduced to:

- prevent accidental use of variables before initialization
- make code more predictable
- avoid subtle bugs caused by undefined values

4.4 Two-Pass Execution Model

The underlying reason for this behavior is that JavaScript works in a **two-pass way**. In the first pass, it scans the code and allocates memory for all declarations. In the second pass, it executes the code line by line.

- var: initialized during the first pass

- let/const: created during the first pass but initialized only during execution

This distinction explains why accessing var yields undefined, while accessing let or const before initialization results in an error.

With a firm grasp of these underlying mechanics, we can now appreciate the modern syntax that was introduced to abstract them away and provide a more predictable development experience.

5 Modern Abstraction: ES6 Classes as Syntactic Sugar

With the introduction of ECMAScript 6 (ES6), JavaScript received a modern, cleaner syntax for implementing object-oriented patterns: the class. While this syntax provides a more familiar structure for developers coming from class-based languages, it is crucial to understand that it is **syntactic sugar**. Under the hood, ES6 classes are built directly on top of JavaScript's existing prototypal inheritance system.

The following example demonstrates the modern class syntax for creating a User:

```
class User {
  constructor(name) {
    this.name = name;
  }
  sayHi() {
    console.log(‘Hi, I’m ${this.name}‘);
  }
}

const user1 = new User(”Charlie”);
user1.sayHi(); // ”Hi, I’m Charlie”
```

5.1 Deconstructing ”Syntactic Sugar”

The term ”syntactic sugar” means that the class keyword does not introduce a new object-oriented inheritance model. Instead, it provides a simpler way to work with prototypes. Internally:

- A class is still a special kind of **function**.
- Methods defined within the class (like sayHi) are automatically added to the class's prototype property.
- Creating an instance with the new keyword sets the instance's ___proto___ to the class's prototype, just as it does with a constructor function.

We can prove this underlying connection by inspecting the User class and its instance:

```
// A class is just a function
console.log(typeof User); // ”function”

// The instance’s ___proto___ points to the class’s prototype, establishing the chain
console.log(user1.___proto___ === User.prototype); // true
```

These checks confirm that the class syntax is a powerful abstraction that ultimately resolves to the same prototypal mechanics we have explored.

In summary, while the modern class syntax simplifies object creation and inheritance, a deep understanding of the underlying prototype chain remains fundamental to becoming a proficient JavaScript developer. The prototype is the engine of JavaScript's object model, and recognizing its role—even when abstracted by modern syntax—is the key to unlocking the language's full potential.

6 The Core Execution Engine: How JavaScript Runs Code

To write predictable and efficient code, it is crucial to understand the fundamental components of the JavaScript runtime—the engine, the heap, and the call stack. While modern JavaScript engines employ sophisticated optimizations, a clear mental model of the core execution process remains indispensable. This section deconstructs what happens "under the hood" when a script is executed, revealing the mechanics that govern everything from variable access to function invocation.

6.1 The JavaScript Runtime Environment

The execution of JavaScript code is a collaboration between two key pieces of software: the **JavaScript Engine** and the **Host Environment**.

- The **JavaScript Engine** is the core component that implements the language, taking source code, parsing it, and executing it. It manages memory and the flow of control.
- The **Host Environment** provides the context in which the engine runs. Environments like a web browser or Node.js offer external, environment-specific APIs that allow JavaScript to interact with the outside world, such as accessing the HTML DOM, making network requests (fetch), or accessing the file system.

6.2 Memory Management: The Heap and The Call Stack

To manage a running program, any language runtime must answer two fundamental questions: "Where do I store my data?" and "What am I currently doing?". In JavaScript, the answers are the Heap and the Call Stack, respectively. JavaScript's memory model is organized into these two primary structures that serve distinct purposes.

- **The Heap:** The heap is a large, unstructured region of memory used for dynamic allocation. When you create objects or define variables in your program, they are stored in the heap. This memory is managed automatically by the JavaScript engine's garbage collector, which frees developers from the need to manually allocate and deallocate memory, as is common in languages like C or C++.
- **The Call Stack:** The Call Stack is a LIFO (Last-In-First-Out) data structure that the JavaScript engine uses to track function execution. When a function is called, a new stack frame, also known as an **Execution Context**, is created and pushed onto the top of the stack. This frame contains information about the function's arguments and local variables. When the function completes its execution and returns, its frame is popped off the stack. JavaScript is inherently single-threaded, which means it has only one call stack and can therefore only execute one operation at a time.

6.3 The Execution Context (EC) and Hoisting

The JavaScript engine executes code by creating an **Execution Context (EC)**. Each script (global code) and each function invocation generates a new execution context. The creation of an execution context happens in two distinct phases.

6.3.1 Global Execution Context

Before execution begins, the engine creates the **Global Execution Context (GEC)**. During its creation:

- A **global object** is created (window in browsers)
- All global variables and functions become properties of this object
- The keyword this is bound to the global object
- Memory is allocated for variables and function declarations

The global object also acts as a container for Web APIs such as setTimeout, XMLHttpRequest, location, and history.

6.3.2 Two Phases of Execution Context Creation

1. **Memory Creation Phase (Hoisting Phase):** During this phase, the engine scans the code without executing it.
   - **Function declarations:** The engine stores a **reference to the entire function** (including its body)
   - var **variables:** Initialized with undefined
   - let **and** const**:** Allocated but left uninitialized (Temporal Dead Zone)

   Important:
   - No code is executed in this phase
   - No function execution contexts are created yet
   - Only bindings (name → value/reference) are established
2. **Execution Phase:** In this phase, the engine executes the code line by line.
   - Variables are assigned their actual values
   - Function declarations are skipped (already in memory)
   - Function calls trigger the creation of new execution contexts

   When a function is invoked:
   - a new **Function Execution Context** is created;
   - it is pushed onto the **call stack**;
   - the memory creation phase runs for that function;
   - the arguments object is created. It stores the values passed to the function at invocation time and includes a length property indicating how many arguments were actually supplied, rather than how many parameters were declared;
   - this is bound (typically to the global object in non-strict mode);
   - the function body is executed.

   Once the function finishes, its execution context is removed from the stack.

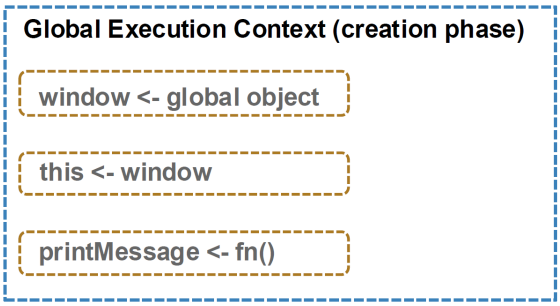

Figure 6.1: Creation of the Global Execution Context

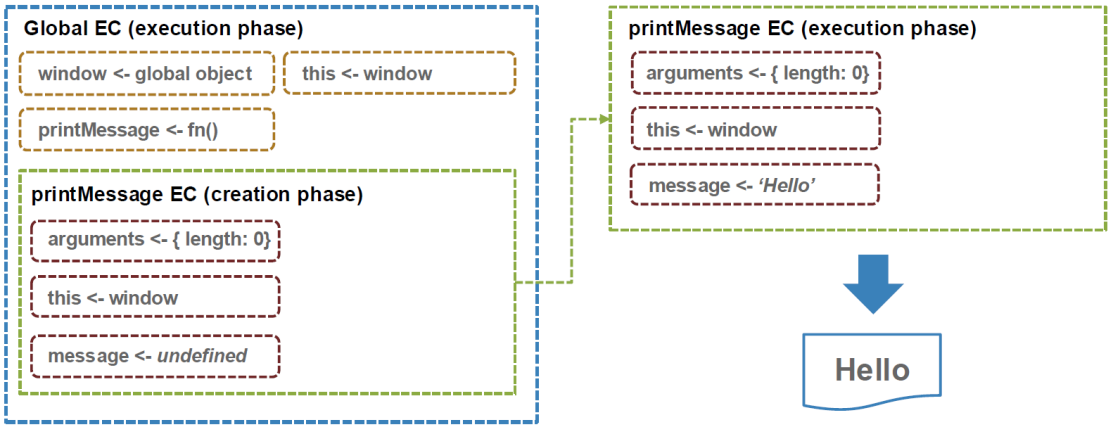

Figure 6.2: Creation of the function execution context and execution phase

6.3.3 Internal Structure of an Execution Context

Each execution context contains:

- **Variable Environment:** stores variables and function references
- **Scope Chain:** used to resolve variable access
- this **binding:** determined at runtime

6.3.4 Example: Memory creation phase and execution phase of a simple print function

Let us say that we have the simple following function that prints the string "Hello":

```
printMessage();

function printMessage(){
    var message = "Hello";
    console.log(message);
}
```

The JavaScript engine runs this code in the following steps:

1. **Creation of the Global Execution Context** (see figure 6.1):
   - The global object is bound to the window
   - The window is bound to the this keyword
   - The engine creates a function reference to the printMessage function
2. **Creation of the printMessage Execution Context** (see figure 6.2):
   - The arguments object is created and contains the arguments passed to the function (in this case, length = 0)
   - The window is bound to the this keyword
   - The JavaScript engine binds the message variable to the value undefined
3. **Execution phase of the printMessage function**
   (1) The engine assigns the value to the message variable
   (2) The engine prints the variable

6.3.5 Visual Model

The following diagram illustrates the difference between the memory phase and the execution phase:

Memory Creation Phase	Execution Phase
Global EC created	Code executed line by line
var x → undefined	x = 10
let y → TDZ	y = 20
foo → function reference	foo() called → new EC created

6.3.6 Call Stack Behavior

Let us make an example of nested function call and illustrate what happens in the stack.

```
function bar(){
    ...
}

function foo(){
    ...
    bar();
    ...
}

// Call of the foo function from the Global EC
foo();
```

Execution contexts are pushed onto the stack when functions are invoked and popped when they return. In fact, in figure 6.3 we can see that at the bottom of the stack we have the global EC. Then, when the function foo is invoked, its EC is put on top of the global one. When the function foo invokes the function bar, the context of the latter is put on top of the stack.

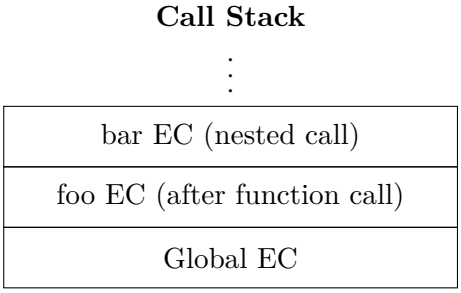

Figure 6.3: Stack behavior during a nested function call

6.3.7 Key Observations

- Hoisting does **not** execute functions
- Functions are stored as **references**, not as execution contexts
- Execution contexts are created **only when functions are invoked**
- The call stack keeps track of the execution order

JavaScript first prepares the environment (memory phase), then executes the code (execution phase).

This synchronous, single-threaded execution model is straightforward but presents a significant challenge: how can JavaScript handle time-consuming operations without freezing the entire application?

7 The Concurrency Model: Event Loop and Queues

JavaScript's concurrency model is the architectural answer to the limitations of its single-threaded nature. Despite having only one call stack, JavaScript masterfully handles asynchronous operations like network requests or timers without blocking the main thread. This non-blocking behavior is not magic; it is achieved through a sophisticated interplay between the JavaScript engine, APIs provided by the host environment, an Event Loop, and a set of distinct job queues (see figure 7.1).

7.1 The Problem with a Single Thread

The limitation of a single call stack is that if a long-running synchronous function is executing, the entire program is blocked. In a browser environment, this means the web page becomes unresponsive; it cannot process user interactions like clicks or scrolls, nor can it render any updates. This is precisely why JavaScript <script> tags are often placed at the end of an HTML document's <body>—it allows the browser to render the page content first before executing potentially blocking scripts.

7.2 The Components of Asynchronous Execution

To enable non-blocking behavior, the JavaScript runtime environment includes several components that work in concert with the engine.

- **Web APIs / Host Environment APIs:** Asynchronous operations are not handled by the JavaScript engine itself. Instead, they are offloaded to APIs provided by the host environment (e.g., the browser or Node.js). Operations like setTimeout, DOM events, and fetch requests are managed by these external APIs. When an operation completes, its associated callback function is prepared for execution.
- **The Macrotask Queue (or Callback Queue):** This is a FIFO (First-In-First-Out) queue where callbacks for completed asynchronous tasks are placed. For example, when a setTimeout timer expires or a user clicks a button, its callback function is added to this queue.
- **The Microtask Queue:** This is a high-priority FIFO queue where callbacks for promises (e.g., .then(), .catch()) and async/await are placed. Microtasks are always processed before any tasks from the Macrotask Queue.
- **The Event Loop:** The Event Loop is the central coordinator. Its primary role is to continuously monitor the Call Stack and the two queues. When the Call Stack is empty, the Event Loop checks the queues and moves tasks to the stack for execution according to a strict order of priority.

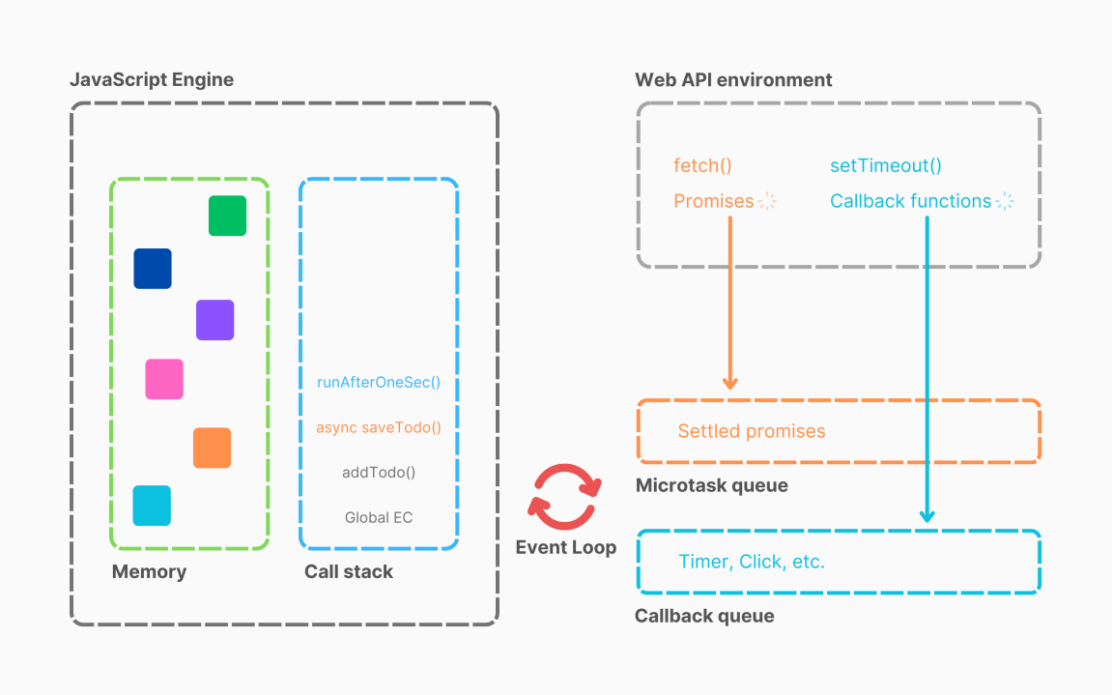

Figure 7.1: JavaScript runtime environment

7.3 The Order of Operations

The event loop manages the precise order of execution with a strict set of rules, ensuring a predictable and non-blocking flow.

1. Execute all synchronous code currently on the Call Stack until it is empty.
2. Check the Microtask Queue. If it has tasks, execute all of them in order until the Microtask Queue is empty.
3. Check the Macrotask Queue. If it has a task, take the first one, push its callback to the Call Stack, and execute it.
4. Repeat the cycle. After the single macrotask completes and the Call Stack is empty again, the Event Loop returns to step 2, always completely draining the Microtask Queue before checking for the next macrotask.

Starvation of macrotasks Although this scheduling model ensures determinism, it may lead to starvation. If microtasks continuously enqueue new microtasks, the Microtask Queue may never become empty. As a result, macrotasks (e.g., setTimeout) may be indefinitely delayed. This behavior is not a flaw but a consequence of the design choice that gives strict priority to microtasks.

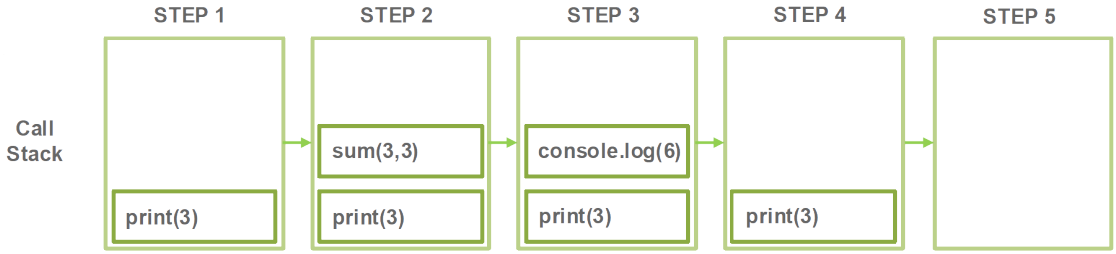

Figure 7.2: Synchronous code execution

7.4 Example of synchronous code execution

Let us expose how the JavaScript runtime environment executes the following code.

```
function sum(x,y){
    return x + y;
}

function print(x){
    var tot = sum(x,x);
    console.log(tot);
}

print(3);
```

This code only contains synchronous functions so, the JavaScript engine will only employ the call stack as depicted in figure 7.2.

The way the call stack works is simple:

- It is initially empty
- When a function is invoked, a stack frame is pushed on top of the stack
- When a function is completed, a stack frame is popped from the top of the stack

In this example, the first invoked function is print(3). It is pushed onto the call stack with the evaluated argument value 3. The newly created execution context of the function print(3) has a dedicated memory space called the **variable environment** (or lexical environment, more on this topic in the following chapters). The variable environment contains the following bindings:

- $x \leftarrow 3$
- $tot \leftarrow undefined$

When the body of the print(3) function invokes the function sum(x,x) it first resolves the value of x (which is 3) from the current execution context and then pushes the instantiated invocation sum(3,3) onto the stack. The lexical scope of the sum(3,3) function contains the following associations:

- $x \leftarrow 3$
- $y \leftarrow 3$

The execution of the sum(3,3) function returns the value 6 which is associated to the variable tot. So, now the lexical scope of the print(3) function contains the following associations:

- $x \leftarrow 3$
- $tot \leftarrow 6$

Now, the body of the print(3) function invokes console.log(6), printing the value to the console. Although console.log is a host function provided by the environment (e.g., the browser or Node.js), its invocation still creates a JavaScript execution context and is managed through the call stack. However, its internal implementation is not executed by the JavaScript engine itself, but is delegated to the host environment.

This execution model also applies to Web APIs such as setTimeout, fetch, and DOM-related functions. The JavaScript engine handles their invocation, but the actual operations (timers, network requests, DOM manipulation) are executed by the host environment. Once completed, their callbacks are placed into the appropriate task queue (e.g., callback queue or microtask queue) to be processed later by the event loop.

Finally, the value 6 is printed in the console, the execution context of the console.log(6) function is popped from the stack, the body of the print(3) function ends and the execution context of the latter is popped from the stack.

Consider that while the call stack is executing synchronous code, the browser cannot perform other tasks, which may lead to blocking behavior and this create concurrency problems. We don't notice this interruption for short task handled using the call stack. For longer tasks, instead, the JavaScript runtime uses the callback queue and the event loop to handle asynchronous operations. The following example exposes an example of callback queue usage.

7.5 Example of asynchronous code execution

Figure 7.3 shows an example of macrotask execution. The key idea is that a function executed in the call stack can delegate work to a Web API, which later schedules a callback in the callback queue (macrotask queue).

It is important to note that when the call stack becomes empty, the event loop first processes all pending microtasks (if any), and only then takes one macrotask from the callback queue and pushes it onto the call stack for execution.

Let us analyze the following code step by step:

```
console.log('10');

setTimeout(function cb() {
    console.log('cb');
}, 2000);

console.log('hi');
```

1. The global execution context starts executing the code.

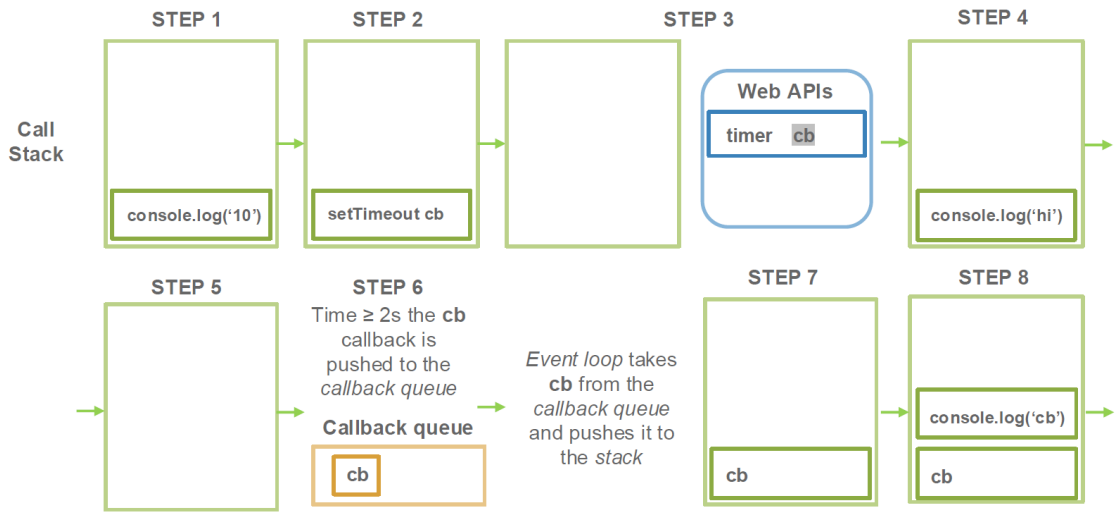

Figure 7.3: Example of macrotask execution

2. console.log('10') is pushed onto the call stack, executed, and then removed. The value 10 is printed to the console.
3. setTimeout(...) is pushed onto the call stack and executed. The JavaScript engine delegates the timer operation to the Web API environment, passing the callback function cb and the delay (2000 ms). The setTimeout call then completes and is removed from the call stack.
4. While the timer is running in the Web API environment, the JavaScript engine continues executing the remaining synchronous code.
5. console.log('hi') is pushed onto the call stack, executed, and then removed. The value hi is printed to the console.
6. After approximately 2000 ms, the Web API finishes the timer and places the callback function cb into the callback queue (macrotask queue).
7. When the call stack becomes empty, the event loop checks the microtask queue and processes all its tasks (if any). Since there are no microtasks in this example, it proceeds to the macrotask queue.
8. The event loop takes the callback cb from the macrotask queue and pushes it onto the call stack.
9. The callback function cb is executed, printing 'cb' to the console, and then removed from the call stack.

This execution model allows JavaScript to remain non-blocking: long-running or delayed operations are handled outside the main thread, while the event loop ensures their callbacks are executed at the appropriate time.

The JavaScript runtime environment is an elegant system that allows JavaScript to remain responsive while handling complex asynchronous flows. The primary tools developers use to manage the state and scope of these asynchronous callbacks are closures and promises.

8 Managing Scope and State: Closures

The concept of scope is fundamental to managing variable accessibility and preventing unintended side effects in a program. JavaScript employs lexical scoping, which determines variable access based on where functions and variables are written in the source code. A powerful and essential consequence of this scoping model is the closure, a mechanism that allows functions to maintain access to their surrounding state.

8.1 Lexical Scoping

Scope defines the context that determines where variables and functions are accessible. JavaScript uses **lexical scoping** (also known as static scoping), meaning the engine determines the scope of variables based on their physical position in the source code, not during execution. A key principle of lexical scoping is that inner scopes have access to variables defined in their outer scopes. This hierarchical structure allows for a clear and predictable organization of code.

Let us analyze the following example:

```
function print(){
  //function scope
  let counter = 0;
  console.log(counter);
}

print(); // 0
console.log(counter); // ReferenceError: counter is not defined
```

In this case we get a reference error because the we are trying to access the variable counter from an outer scope. We can consider it as an advantage because this allows us to have variables with the same name in different scopes. For instance:

```
function print(){
  //function scope
  let counter = 0;
  console.log(counter);
}

print(); // 0

let counter = 10;
console.log(counter); // 10
```

In this case we have two variables named counter and the interpreter does not raise an error because it understands that the counter used in the print function scope refers to the variable initialized with a value of 0 while the counter used in the outer scope refers to the one initialized with a value 10.

8.2 Defining a Closure

A **closure** is a function that is bundled with its surrounding state—the lexical environment. In practical terms, a closure is a nested function that "remembers" the variables from the scope where it was created (the one of the outer function), even if it is executed in an entirely different scope at a later time. This is possible because the inner function maintains a live reference to the *Execution Context* of its parent—specifically, its lexical environment—even after the parent function's EC has been popped from the call stack.

8.3 Practical Application of Closures

Closures are not merely a theoretical concept; they are a powerful tool for creating private state that persists across multiple function calls. Consider the following counter example:

```
function outerFunction(){
    // Outer scope
    var counter = 0;

    function innerFunction(){
        // Inner scope
        console.log(counter);
        counter += 1;
    }

    return innerFunction;
}

const myInnerFunction = outerFunction();
myInnerFunction(); // 0
myInnerFunction(); // 1
myInnerFunction(); // 2
```

In this code, outerFunction is invoked, which initializes the counter variable to 0. It then returns innerFunction, which is assigned to the constant myInnerFunction. Because innerFunction is a closure, it maintains a persistent reference to the lexical environment where counter lives. Each time myInnerFunction is invoked, it can access and modify this same counter variable. The counter variable is effectively private, as it cannot be accessed from the global scope, yet its state is preserved between calls.

Crucially, outerFunction is invoked *only once*. If it were called each time, the counter would be reset to 0 with every call. By storing the returned innerFunction in myInnerFunction, we preserve the closure's lexical environment, allowing the counter state to persist across multiple calls.

This ability to "remember" state is also foundational to how Promises work; the executor function passed to a Promise constructor relies on a closure to access variables from its surrounding context.

9 Asynchronous Operations with Promises

A Promise is a foundational object in modern JavaScript for managing asynchronous operations. It serves as a proxy for a value that will be available in the future, allowing asynchronous code to be written in a more manageable, readable, and composable way than with traditional callback-based approaches. It represents the eventual completion or failure of an asynchronous task and its resulting value.

9.1 The Promise Object and its States

The Promise object can exist in one of three distinct states:

- pending: The initial state. The asynchronous operation has not yet completed.
- fulfilled: The operation completed successfully, resulting in a value.
- rejected: The operation failed, resulting in a reason (typically an error object).

A promise is considered **settled** when it is either fulfilled or rejected. Once settled, a promise is immutable and its state cannot change.

9.2 Creating and Consuming Promises

9.2.1 Creating a Promise

A promise is created using the new Promise(executor) constructor. The executor is a function that runs immediately and is passed two arguments: resolve and reject. These are callback functions provided by the JavaScript engine.

- resolve(value): Called when the asynchronous operation completes successfully.
- reject(reason): Called when the operation fails.

The executor's role is to initiate an asynchronous operation and call the appropriate callback to settle the promise. Because the executor function cannot be passed custom arguments, it relies on a **closure** to capture and access variables from its surrounding scope.

```
var promise = new Promise(function(resolve, reject){
    // let state = "success";
    let state = "network error";

    if (state != "success"){
        // The argument of the reject callback is the failure message
        reject("The operation was not successful");
    } else {
        // The argument of the resolve callback is the success message
        resolve("The operation WAS succesful");
```

```
    }
});
```

9.2.2 Consuming a Promise

The eventual result of a promise is handled by attaching callbacks using the .then() and .catch() methods. For example, the previously defined promise can be handled as follows:

```
promise.then(
  function (successMessage){
    console.log(successMessage);
})
.catch(
  function (failureMessage){
    console.log(failureMessage);
});
```

- .then(onFulfilled): The onFulfilled handler is called when the promise is fulfilled and receives as an argument the data provided in the resolve callback in the promise.
- .catch(onRejected): The onRejected handler is called when the promise is rejected and receives as an argument the data provided in the reject callback in the promise.

Crucially, both .then() and .catch() return a **new promise**, which is the key to creating clean, sequential asynchronous workflows through chaining.

9.3 Promise Chaining and async/await

Promise chaining is the practice of executing multiple asynchronous operations in sequence. To properly chain promises, it is critical to ***return*** a value or another promise from within a .then() handler. If a promise is not returned, the next .then() in the chain will execute immediately with a value of undefined, leading to a ”floating promise” that cannot be tracked.

Correct promise chaining In a correct chain, each .then() returns a value or a new promise:

```
function step1() {
    return Promise.resolve(”Step 1 completed”);
}

function step2(prevResult) {
    return Promise.resolve(prevResult + ” -> Step 2 completed”);
}

step1()
.then(function(result){
    console.log(result);
    return step2(result);
})
.then(function(result){
    console.log(result);
});
```

Output:

```
Step 1 completed
Step 1 completed -> Step 2 completed
```

Floating promise (incorrect chaining) If a promise is not returned, the chain breaks:

```
step1()
.then(function(result){
    console.log(result);
    step2(result); // NOT returned
})
.then(function(result){
    console.log(result); // undefined
});
```

In this case, the second .then() does not wait for step2() to complete, because its promise is not returned. This leads to loss of control over the asynchronous flow.

Using async/await Modern JavaScript provides async/await, which is syntactic sugar built on top of promises. It allows asynchronous code to be written in a sequential style.

```
async function runSteps() {
    let result1 = await step1();
    console.log(result1);

    let result2 = await step2(result1);
    console.log(result2);
}

runSteps();
```

Important notes:

- await can only be used inside async functions
- await pauses the function execution (non-blocking) until the promise is resolved
- under the hood, async/await is still based on promises and the microtask queue

These foundational patterns enable developers to manage simple asynchronous tasks, but real-world applications often require more sophisticated patterns for handling multiple concurrent operations and complex error scenarios.

9.4 Advanced Promise Patterns

While basic promise chaining is sufficient for many tasks, modern applications often require coordinating multiple asynchronous operations concurrently or implementing sophisticated error recovery logic. Advanced promise patterns and static methods provide the tools needed to manage these complex scenarios gracefully.

9.4.1 Concurrent Operations

The Promise object includes four static methods for handling concurrent operations. Each method takes an iterable of promises and returns a single promise whose behavior depends on the outcomes of the input promises.

Method	Behavior
Promise.all()	Fulfills when **all** of the input promises have fulfilled, with an array of their fulfillment values. It rejects immediately if **any** of the input promises reject.
Promise.allSettled()	Fulfills when **all** input promises have settled (either fulfilled or rejected), with an array of objects describing the outcome of each promise.
Promise.any()	Fulfills as soon as **any** of the input promises fulfill, with that promise's fulfillment value. It rejects only if **all** of the input promises reject.
Promise.race()	Settles (fulfills or rejects) as soon as **any** of the input promises settle, adopting the outcome of that first promise.

9.4.2 Advanced Error Handling

A robust application must handle errors gracefully. Promises offer flexible patterns for error recovery. The standard approach is to place a single .catch() at the end of a promise chain. This acts as a global handler, catching any rejection that occurs in any of the preceding .then() blocks.

For more precise error recovery, promise chains can be nested. An inner .catch() handler can catch failures from a specific part of the chain, allowing the program to handle and recover from non-critical failures while still allowing the outer chain to proceed. Critical failures from higher up in the chain will bypass the inner catch and be caught by the final, outer handler.

These advanced patterns provide the control flow necessary to build complex, resilient applications, shifting the focus from managing asynchronicity to structuring the code itself.

9.5 Example: Call Stack, Microtasks, and Macrotasks

The following example illustrates how the JavaScript runtime environment coordinates the execution of synchronous code, microtasks (Promises), and macrotasks (Web APIs such as setTimeout).

```
console.log("Start");

setTimeout(() => {
    console.log("Macrotask (setTimeout)");
}, 0);

Promise.resolve()
    .then(() => {
        console.log("Microtask 1 (Promise)");
    })
    .then(() => {
```

```
        console.log("Microtask 2 (chained Promise)");
    });

console.log("End");
```

9.5.1 Expected Output

```
Start
End
Microtask 1 (Promise)
Microtask 2 (chained Promise)
Macrotask (setTimeout)
```

9.5.2 Step-by-Step Execution

1. **Synchronous Execution (Call Stack)**
   1.1. console.log("Start") is pushed to the call stack and executed.
   1.2. setTimeout(...) is invoked and delegated to the Web APIs environment.
   1.3. The Promise is resolved immediately, and its .then() callbacks are placed in the **Microtask Queue**.
   At this point, one might think that between the Promise.resolve() instruction and the console.log("End") instruction the call stack is empty so the JavaScript engine would run the microtasks. Actually not. The JavaScript engine considers as synchronous code all the code in the main script. After the main script finishes it checks for the microtask queue and the macrotask queue so, the right mental model is:
   $Main\ script \rightarrow [Empty\ call\ stack] \rightarrow Microtask\ queue \rightarrow Macrotask\ queue$
   1.4. console.log("End") is executed.

   The call stack is now empty.
2. **Microtask Queue Execution**
   2.1. The event loop processes the Microtask Queue.
   2.2. "Microtask 1" is moved from the queue to the call stack and executed.
   2.3. The chained .then() adds another microtask in the queue. This is moved from the queue to the stack and executed immediately after.

   After all microtasks are completed, the event loop checks the Macrotask Queue.
3. **Macrotask Queue Execution**
   3.1. The callback from setTimeout is moved to the call stack.
   3.2. "Macrotask (setTimeout)" is printed.

9.5.3 Key Observations

- Microtasks (Promises) have higher priority than macrotasks.
- All microtasks are executed before any macrotask.
- Even with a delay of 0 ms, setTimeout does not execute immediately.
- The event loop ensures that the call stack is empty before processing queued tasks.

10 Structuring Code: Object Creation Patterns

10.1 Foundations of Design Patterns in JavaScript

In the discipline of software architecture, a "pattern" is analytically defined as a recurring, high-level solution to a common design problem. These are not proprietary algorithms, but rather structured blueprints that have been distilled from repeated implementations across diverse software systems. The lineage of this conceptual framework traces back to the architect Christopher Alexander's work in urban design and was formally introduced to software engineering by the "Gang of Four" (Gamma, Vlissides, Johnson, and Helm) in 1994. In the context of large-scale JavaScript development, patterns are strategic tools used to manage the inherent complexity of a language that lacks the rigid, static structures of traditional class-based environments.

Object Creation Patterns are specialized architectural structures designed to handle the complexities of instantiation. Unlike class-based languages that rely on formal "blueprints," JavaScript's prototype-based nature is fundamentally more flexible and "wild," focusing on the relationships between live objects. This flexibility necessitates a disciplined approach to how objects are created and linked. Historically, the evolution of these patterns was driven by a dual strategic requirement: the need for memory efficiency through shared behavior and the demand for robust state encapsulation.

10.2 The Object Decorator Pattern

The Object Decorator—often referred to as a "Wrapper"—is a structural pattern that evaluates the extension of object functionality at runtime. From an architectural standpoint, it favors aggregation and composition over static inheritance. This allows developers to avoid the "combinatorial explosion" of subclasses that occurs when attempting to model every possible variation of a component at design time.

A useful analogy for junior developers is that of clothing: a base entity (the person) can wrap themselves in a sweater, then a jacket, and finally a raincoat. Each layer extends the core "weather resistance" behavior without altering the underlying person or requiring a new "RainyWeatherPerson" subclass. In implementation, a decorator function accepts an existing object and augments it with new properties or methods, often wrapping existing logic to add pre- or post-processing capabilities.

This strategy is implemented creating a function that accepts a target object as a parameter, adds properties/methods (wrapping original logic), and returns the augmented instance.

- **Pros:**
  - Dynamic composition;

  - Adheres to the Open/Closed Principle;
  - Avoids subclass bloat;
  - Allows for runtime behavior modification.
- **Cons:**
  - High memory overhead;
  - Creates significant **Garbage Collection (GC) pressure** due to unique function objects;
  - Difficult to track source logic;
  - Potential for name collisions.
- **Typical Use Cases:**
  - Middleware systems (e.g., Express.js);
  - Dynamic UI feature flags;
  - Plugin architectures where third-party extension is required.

10.2.1 Example of decorator pattern usage

Let us make an example in which we want to turn a generic object into a car. In this case we create a decorator function that takes in input the generic object and add the position property and the move function. Let us make a first example in which we only add the position property and create the move function externally to the decorator function.

```
var buildcar = function(obj, position){
    obj.position = position;
    return obj;
};

var move = function(car){
    car.position++;
};

var carA = buildcar({}, 1);
move(carA);
```

buildcar is our decorator function which takes the object in input, adds the position property and returns the decorated object. In the final part of the snippet we use the decorator function on the empty object ({}) and we use the move function on the object returned by the decorator function. The schematization of the first technique is illustrated in figure 10.1.

Using an external function is more memory efficient because decorated objects are lighter but it exposes a public function. This may introduce a vulnerability in a context in which we want strong encapsulation and true private state. For this reason, the decorator pattern also allows us to add functions to objects. In the following example we use this other technique.

```
var buildcar = function(obj, position){
    obj.position = position;
    obj.move = function(){
        // obj always refers to the same object
        obj.position++;
```

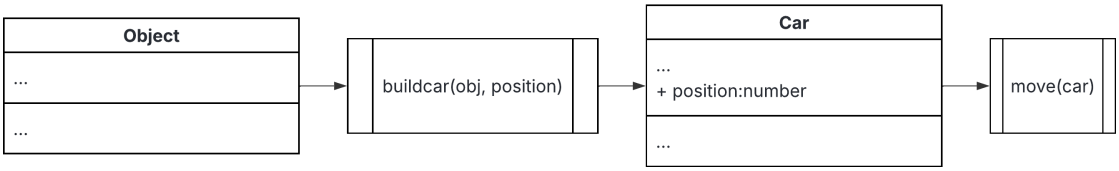

Figure 10.1: Example of usage of the decorator function to add only an attribute to an object

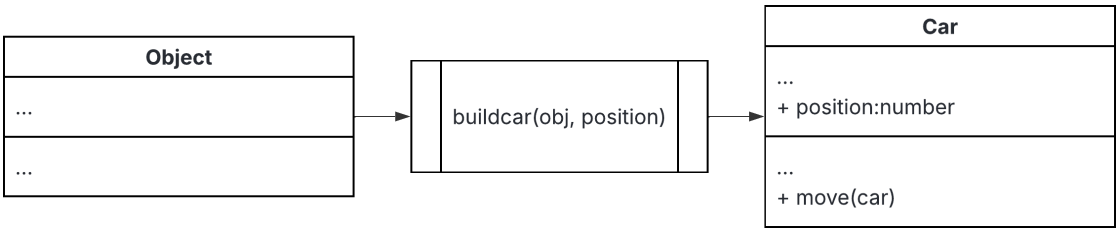

Figure 10.2: Example of usage of the decorator function to add an attribute and a method to an object

```
    }
    return obj;
};

var carA = buildcar({}, 1);
carA.move();
```

In this case, the buildcar function creates a closure scope and the obj variable inside the function will always point to the same object. This is beneficial for encapsulation and security but makes the object heavier because all objects will contain their own function.The schematization of the second technique is illustrated in figure 10.2.

While decorators provide runtime flexibility, the systemic shift from augmenting existing instances to creating new ones from a template led to the development of the "Functional Class" pattern.

10.3 Functional Classes

The Functional Class pattern was an early attempt to create standardized "molds" for similar objects using constructor functions. It aimed to automate the reproduction of objects sharing the same "type"—possessing identical state structures and methods—while maintaining strong encapsulation through closures.

In Functional Classes, unlike in the Decorator Pattern, the **constructor function** (the function that produce fleets of similar objects) initializes an internal object literal, assigns state and methods to it using passed arguments, and returns that object. This pattern is often chosen to avoid the complexities of the this keyword and the associated risk of **context loss** (where this inadvertently points to the global object).

Performance Implications of Method Duplication From an architectural perspective, the "So What?" of this pattern is its inherent memory inefficiency. Because every instance created via a functional class carries a unique copy of its methods, memory consumption grows linearly with the number of instances. This creates high **GC Pressure**, as the JavaScript engine must periodically spike CPU usage to clean up thousands of identical function objects that could have been shared.

- **Pros:**
  - Strong encapsulation via closures (true private state);
  - Avoids this keyword complexity;
  - Intuitive syntax.
- **Cons:**
  - Highly memory inefficient;
  - Fails the instanceof test because Object.getPrototypeOf() returns a generic Object.prototype;
  - Stripping the object of its architectural identity.
- **Use Cases:**
  - Scenarios requiring absolute state privacy;
  - Small-scale utilities where performance is secondary to security.

10.3.1 Example of functional class usage

Let us make the car object example using this other pattern.

```
var Car = function(position){
    var obj = {position: position};
    obj.move = function(){
        obj.position++;
    }
    return obj;
};

var carA = Car(1);
carA.move();
```

Using this pattern, the function does not accept a generic object in input which could have attributes and methods, instead, it creates the object and the assigns it its own function. The schematization of this example is depicted in figure 10.3.

10.4 Functional shared pattern

The functional shared pattern is a variant of the functional class pattern in which object functions are defined externally to the object and the constructor function assigns methods to the objects as references. Defining functions externally is more memory efficient but introduces the complexity related to contexts and the this keyword.

Let us understand this pattern better by exposing an example.

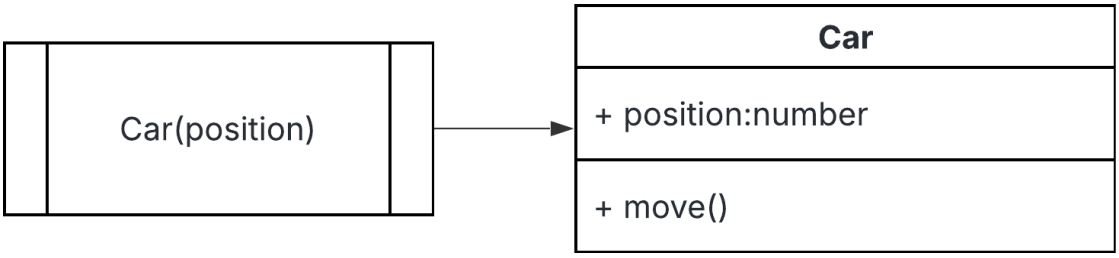

Figure 10.3: Example of functional class usage

10.4.1 Example of functional shared pattern usage

Let us make the car object example as we did using the functional class pattern but defining the move function externally:

```
var Car = function(position){
    var obj = {position: position};
    obj.move = move;
    return obj;
};

var move = function(){
    // The move function cannot access obj anymore
    obj.position++;
};
```

This implementation will not work because the move function no longer has closure scope access to the obj variable. We can use this pattern if we implement the move function using the this keyword to refer to the car object.

```
var Car = function(position){
    var obj = {position: position};
    obj.move = move;
    return obj;
};

var move = function(){
    this.position++:
};

var carA = Car(1);
carA.move();
```

This implementation will work because the move function refers to the car object through the this keyword. Notice that the reference to the move function inside the car object is defined as: obj.move = move;. To the left of the move function there is obj which means that the this keyword inside the externally defined move function will refer to obj. The schematization of this example is depicted in figure 10.4.

Roughly speaking: the functional shared pattern is a variant of the functional classes pattern in which we define methods externally to the objects to trade off memory efficiency for context

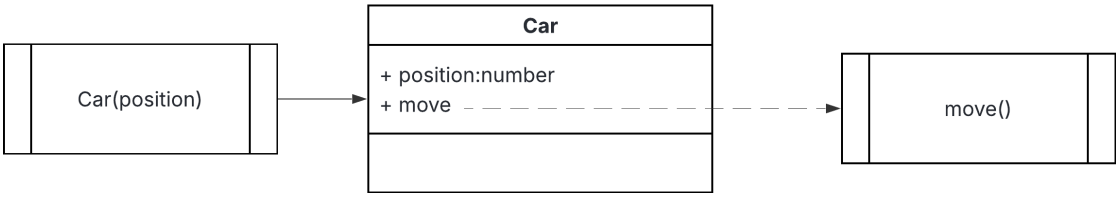

Figure 10.4: Example of functional shared pattern usage

management complexity.

At this point one might say: what if we want a trade-off between the functional classes pattern and the functional shared pattern? What if, given a class of objects, we only need true encapsulation for some of them while we can afford external definition and memory efficiency for the others? This reasoning is the foundation of the following pattern.

10.5 Functional Shared Pattern with Closure (Hybrid Pattern)

The functional shared patter with closure is a hybrid between the latter two patterns. Given an object we establish which ones must be encapsulated for privacy and define externally all the others for memory efficiency. Let us make an example of hybrid pattern usage.

10.5.1 Example of hybrid pattern usage

Let us say that we want to make a car and a horse object. They both move but only the car can honk and only the horse can neigh. For this reason, we can define the move methods externally and the other two internally.

```
var Car = function(position){
  var obj = {position: position};
  obj.move = move;

  obj.honk = function(){
    console.log("I'm honking the horn!");
  }

  return obj;
}

var Horse = function(position){
  var obj = {position: position};
  obj.move = move;

  obj.neigh = function(){
    console.log("Neigh!");
  }

  return obj;
}
```

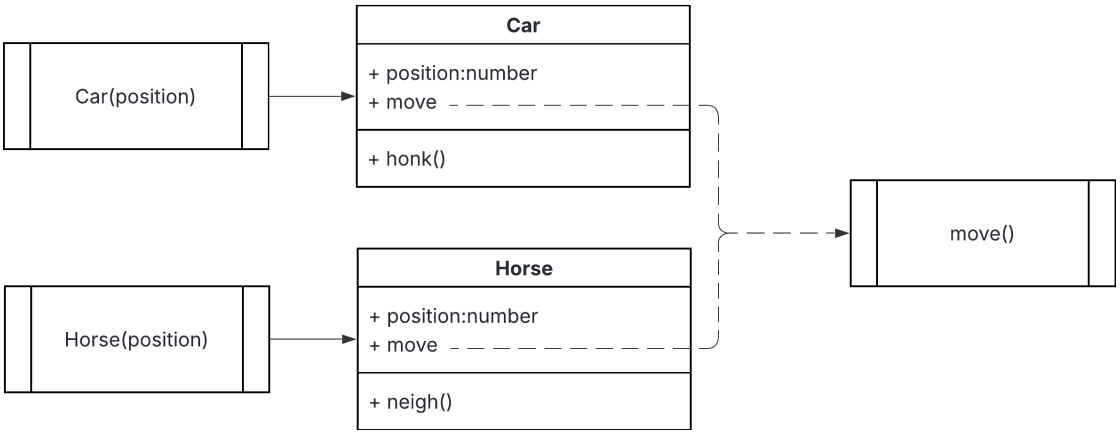

Figure 10.5: Example of hybrid pattern usage

```
var move = function(){
  this.position++;
};

var carA = Car(1);
var horse = Horse(1);

carA.move();
horse.move();

carA.honk();
horse.neigh();
```

In this example in particular we decided to implement methods internally or externally according to a pertinence criterion but in a real life scenario this decision can also depend on privacy and security.

The schematization of the example is depicted in figure 10.5.

10.6 Prototypal Classes

Prototypal classes represent a fundamental shift from "copying" to "delegation." In this model, objects are linked to a central repository of methods rather than carrying their own. The implementation uses Object.create(prototypeObject) to link the new instance's internal [[Prototype]] (known in industry jargon as **dunder proto** or __proto__) to a "mold" object.

The Architectural "Lookup Chain" When a method is called on an instance, the engine follows a specific lookup chain: Instance => Mold Object (__proto__) => Object.prototype => null. For example, carA => Car.methods => Object => null. This delegation is vastly superior for memory management, as a single copy of a method can serve thousands of instances.

- **Pros:**

  - Extreme memory efficiency;
  - Allows "live updates" where modifying the prototype affects all instances immediately.
- **Cons:**
  - More complex syntax;
  - Requires manual state initialization (often through a secondary init method).
- **Use Cases:**
  - Performance-critical systems;
  - Applications managing massive entity counts.

10.6.1 Example of prototypal class usage

```
var Car = function(position){
  var obj = Object.create(Car.methods);
  obj.position = position;
  return obj;
};

Car.methods = {
  move: function(){
    this.position++;
  }
};

var carA = Car(1);
carA.move();

// Prototype chain test
console.log(carA.___proto___); // Car.methods
console.log(carA.___proto___.___proto___); // Object
console.log(carA.___proto___.___proto___.___proto___); // null
```

In this example we have a constructor function which creates obj by assigning the object Car.methods as its prototype. The move function, which is common to all the objects, is assigned to the Car.methods object to leverage the prototype chain. This strategy avoids copying the function saving space and time.

At the end of the code, we can see the prototype chain in action. The schematization of this pattern is depicted in figure 10.6.

10.6.2 Example of prototypal class usage with official conventions

Since this pattern is so common, the language designer decided to add official conventions to support it. Instead of creating a new object (Car.methods in the previous example), we can leverage the default existing one which is called Car.prototype. The prototype object comes by default when we create an object and has a constructor property which stores a pointer to the function it comes attached to which, in our example, is Car.

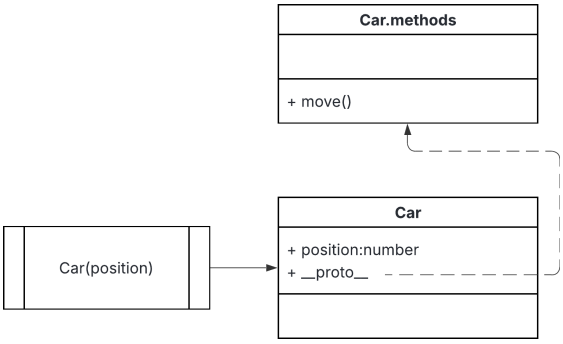

Figure 10.6: Example of prototypal classes usage

```
var Car = function(position){
  var obj = Object.create(Car.prototype);
  obj.position = position;
  return obj;
};

Car.prototype.move = function(){
  this.position++;
};

var carA = Car(1);
carA.move();

// Prototype chain test
console.log(carA.___proto___); // Car.prototype
console.log(carA.___proto___.___proto___); // Object
console.log(carA.___proto___.___proto___.___proto___); // null

// Constructor test
console.log(Car.prototype.constructor); // Car
```

This code is basically the same as the one of the previous example but we use the default object instead of creating a new one. At the end of the snippet there is also a constructor test to check if the constructor property of the prototype object points back to the Car object. The schematization of this pattern is depicted in figure 10.7.

10.7 The Pseudoclassical Pattern

The Pseudoclassical pattern is the standard bridge between traditional class-based expectations and JavaScript's prototypal reality. It uses constructor functions and the new keyword to provide an interface familiar to those from Java or C++ backgrounds while leveraging the efficiency of prototypes.

The new keyword in front of a function invocation triggers the **Constructor Mode**. In this mode the JavaScript engine executes four precise operations:

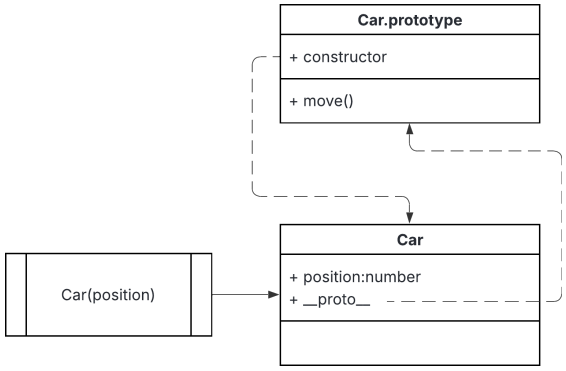

Figure 10.7: Example of prototypal classes usage with official convention

1. **Creation:** A new empty object {} is created.
2. **Linking:** The object's [[Prototype]] is linked to the constructor function's .prototype property.
3. **Binding:** The execution context (this) is bound to the new object.
4. **Return:** The object is implicitly returned.

These operations are abstracted for syntactic convenience considering that they are recurrent and standardized in the prototypal class pattern.

JIT Optimization and Predictability Modern JIT engines like V8 are specifically optimized for this pattern. Because the structure is predictable, the engine can optimize method lookups and property access more effectively than with the "strange" or dynamic structures found in functional patterns. Predictability is the primary driver of performance in high-scale JavaScript.

- **Pros:**
  - Standardized;
  - Memory-efficient;
  - Compatible with instanceof;
  - Highly optimized by V8.
- **Cons:**
  - Fragile if new is omitted (context loss);
  - Syntax for manual prototype attachment is verbose.
- **Use Cases:** General application logic and framework development.

10.7.1 Example of pseudoclassical pattern usage

```
var Car = function(position){
  this.position = position;
};
```

```
Car.prototype.move = function(){
  this.position++;
};

var carA = new Car(1);
carA.move();

console.log(carA instanceof Car); // true
```

This is the implementation of our car example using the pseudoclassical pattern. Notice that in the constructor function there aren't the empty object creation and the return statement returning the object because these operations are abstracted by the JavaScript engine.

At the end of the snippet there is a test which verifies that the newly created object is considered an instance of the Car class.

10.8 Superclasses and Subclasses

Inheritance establishes "is-a" relationships, allowing a subclass to inherit the logic of a parent superclass. This requires a two-pronged strategy:

1. **Constructor Stealing:** The subclass invokes the superclass constructor to initialize base state on the subclass instance.
2. **Behavior Delegation:** The subclass links its prototype to the superclass prototype to inherit methods.

10.8.1 Example of superclasses and subclasses usage

Let us say that we have two constructor functions for the Van class and for the Cop class defined as follows:

```
var Van = function(position){
    var obj = {position: position};
    obj.move = function(){ obj.position++; };
    obj.grab = function(){/*...*/};
    return obj;
};

var Cop = function(position){
    var obj = {position: position};
    obj.move = function(){ obj.position++; };
    obj.call = function(){/*...*/};
    return obj;
};
```

These two constructor functions have duplicate code. This not only makes the constructor functions more complicated and heavier than they could be but also creates maintenance issues because if we want to update the common part of both functions we would have to make the same changes to both the functions. This process is error prone and can be avoided by creating

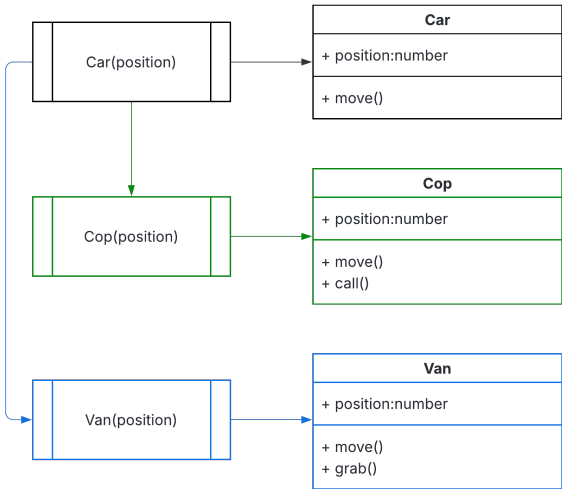

Figure 10.8: Example of superclasses and subclasses usage

a superclass containing the common code of both functions. After we create a superclass, we can use the object it creates as a starting point for the subclasses which will have to implement only their unique codes.

In this example in the specific we can define a Car class to implement the common code and simplify the other classes as follows.

```
var Car = function(position){
    var obj = {position: position};
    obj.move = function(){
        obj.position++;
    };
    return obj;
};

var Van = function(position){
    var obj = Car(position);
    obj.grab = function(){/*...*/};
    return obj;
};

var Cop = function(position){
    var obj = Car(position);
    obj.call = function(){/*...*/};
    return obj;
};
```

The schematization of this pattern is depicted in figure 10.8.

10.9 Pseudoclassical Subclasses

Implementing subclasses pseudoclassically is technically demanding, as missing a single step in the boilerplate can break the inheritance chain or lead to identity errors.

The implementation requires three critical steps:

1. **Invoke Super:** Use Superclass.call(this [, args]) inside the subclass constructor.
2. **Link Prototypes:** Set Subclass.prototype = Object.create(Superclass.prototype).
3. **Reset Constructor:** Manually set Subclass.prototype.constructor = Subclass. This is the most common point of failure; without this reset, the constructor property will incorrectly point to the superclass.

- **Pros:**
  - Full inheritance support;
  - High memory efficiency;
  - Clear type checking.
- **Cons:**
  - Verbose boilerplate;
  - Error-prone manual linking.
- **Use Cases:** Hierarchical domain models (e.g., Vehicle → Car).

10.9.1 Example of pseudoclassical subclasses usage

Pseudoclassical pattern problems analysis

Let us start from the analysis of the problems of the pseudoclassical pattern.

```
var Car = function(position){
  this.position = position;
};

Car.prototype.move = function(){
  this.position++;
};

var Van = function(position){
  // Without the keyword 'new' obj is undefined
  var obj = new Car(position);

  obj.grab = function(){
    console.log("I'm grabbing!");
  }

  // If we don't return the object, vanA is undefined.
  return obj;
};
```

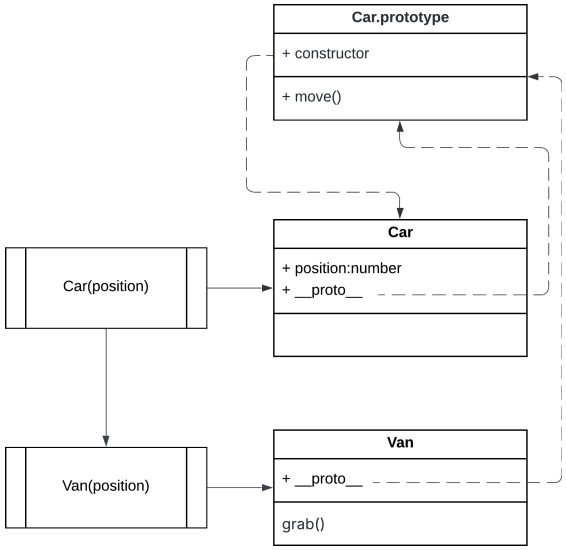

Figure 10.9: Example highlighting the problems of the pseudoclassical pattern

```
var vanA = new Van(3);
vanA.move();
vanA.grab();

console.log(vanA.___proto___ === Van.prototype); // false
console.log(vanA.___proto___ === Car.prototype); // true
console.log(vanA instanceof Van); // false
console.log(vanA instanceof Car); // true
```

This approach requires us to pay particular attention to the boilerplate of the code rather than the business logic. In fact, skipping the new or return keyword causes the code not to work as described in the comments of the code. Furthermore, the prototype chain is deceitful because one might think that the prototype of vanA is Van.prototype and vanA is an instance of Van. Instead, the Van object is decorated Car object.

The schematization of this implementation is depicted in figure 10.9.

Setting the execution context using the Superclass.call function

We can partially address the problems of the pseudoclassical pattern by using Superclass.call(this [, args]) to explicitly control the execution context of the superclass constructor. In this case, we invoke Car in the context of the object being created by Van which means that we execute the constructor of Car in the context of the current object Van.

```
var Car = function(position){
  this.position = position;
};

Car.prototype.move = function(){
```

```
  this.position++;
};

var Van = function(position){
  Car.call(this, position);

  this.grab = function(){
    console.log("I'm grabbing!");
  }
};

var vanA = new Van(3);
// vanA.move(); // ERROR
vanA.grab();
console.log("Van position: " + vanA.position);

var carA = new Car(1);
carA.move();
// carA.grab(); // ERROR
console.log("Car position: " + carA.position);

console.log(vanA.__proto__ === Van.prototype); // true
console.log(vanA.__proto__ === Car.prototype); // false
console.log(vanA.__proto__ === Car); // false
console.log(vanA instanceof Van); // true
console.log(vanA instanceof Car); // false
```

When executing this code, we encounter two errors. The first one occurs at vanA.move();. Although we invoked the superclass constructor using Car.call(this, position), this operation only initializes the current object (i.e., the instance created by new Van) by assigning its properties. It does not establish any link between Van.prototype and Car.prototype. As a consequence, the method move, which is defined on Car.prototype, is not accessible from vanA. In other words, call allows us to reuse the constructor logic of the superclass, but it does not implement inheritance. The prototype chain remains unchanged, and therefore Van is not yet a subclass of Car.

The second error occurs at carA.grab();. This highlights another important aspect: methods defined inside the Van constructor (such as grab) are assigned directly to instances of Van and are not shared with instances of Car. Even though Car.call is used within Van, it does not introduce any relationship in the opposite direction. Therefore, Car instances cannot access methods defined in Van.

Finally, the prototype tests confirm the situation. The object vanA is correctly recognized as an instance of Van, and its internal __proto__ points to Van.prototype. However, since there is no delegation from Van.prototype to Car.prototype, vanA is not an instance of Car and cannot access its prototype methods.

The schematization of this implementation is depicted in figure 10.10. The figure highlights that the two objects and their relative prototype chains are separated and there isn't an inheritance relationship between Van and Car.

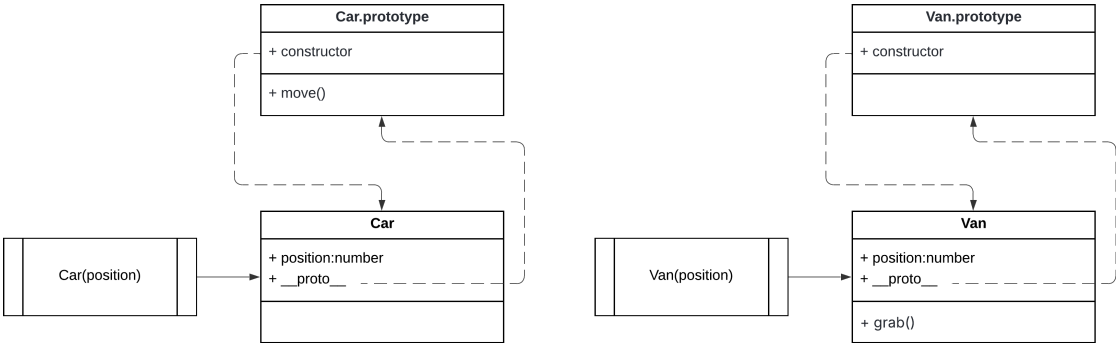

Figure 10.10: Example of Superclass.call function usage to partially solve the problem

Fixing the prototype chain

We can ultimately solve the problem by fixing the prototype chain manually. We need to first create a prototype object for Van and then set the constructor pointer of the newly created object to the Van class.

```
var Car = function(position){
  this.position = position;
};

Car.prototype.move = function(){
  this.position++;
};

var Van = function(position){
  Car.call(this, position);

  this.grab = function(){
    console.log("I'm grabbing!");
  }
};

/*
  We create a new prototype object for Van that delegates to Car.prototype.
  This does NOT copy Car.prototype, but links the two prototype chains.
*/
Van.prototype = Object.create(Car.prototype);

console.log(Van.prototype.constructor); // Car

// We set the constructor pointer to the subclass
Van.prototype.constructor = Van;
console.log(Van.prototype.constructor); // Van

var vanA = new Van(3);
vanA.move();
```

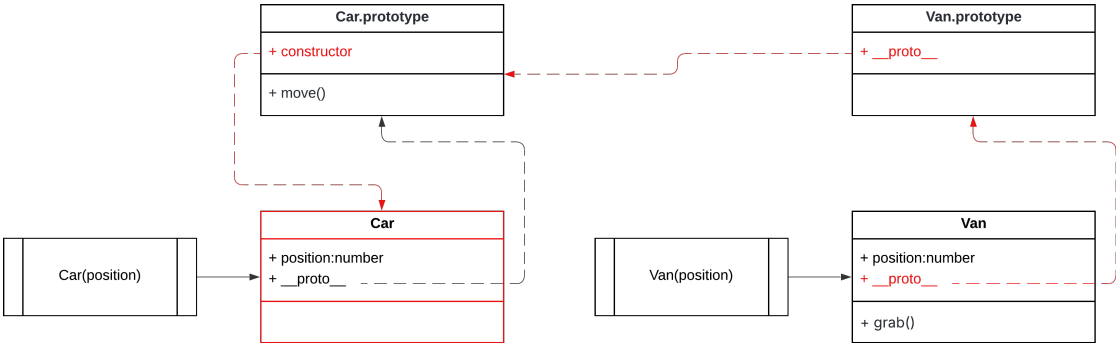

Figure 10.11: Intermediate problematic state of the pseudoclassical subclasses pattern

```
var carA = new Car(1);
carA.move();

console.log(vanA.___proto___ === Van.prototype); // true
console.log(vanA.___proto___ === Car.prototype); // false
console.log(vanA.___proto___ === Car); // false
console.log(Van.prototype === Car.prototype); // false
console.log(Van.prototype.___proto___ === Car.prototype); // true

console.log(vanA instanceof Van); // true
console.log(vanA instanceof Car); // true
```

The reason why we use the Van.prototype = Object.create(Car.prototype); is clear: we want to implement prototypal inheritance between Van and Car. But why do we need to set the constructor of the newly created prototype back to the subclass? Doesn't Van.prototype has a constructor variable that points to Van by default?

If we leave Van.prototype unchanged, it has a constructor variable that points to Van by default. But when we use the create function we completely replace the default object it used to be and we have an object which only has a pointer to its prototype.

```
Van.prototype = {
  ___proto___: Car.prototype
}
```

This intermediate and unintended state is depicted in figure 10.11.

When we call Van.prototype.constructor, the object cannot find the property we are looking for and the request is passed to its prototype i.e. Car.prototype. This object has the constructor property pointing to Car and therefore returns this information. We don't want this to happen so we have to add the constructor property to the newly created Van.prototype object and set it to Van.

The final state working as intended is schematized in figure 10.12.

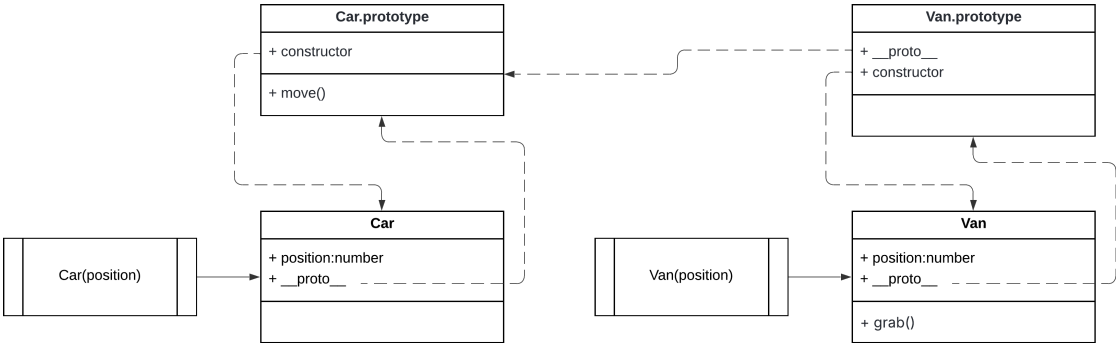

Figure 10.12: Final state of the pseudoclassical subclasses pattern

10.10 The ES6 Class-based Pattern

The ES6 Class-based pattern introduces a higher-level abstraction over JavaScript's prototypal inheritance model. It uses the class syntax along with the extends and super keywords to provide a more familiar and structured way to define classes and subclasses.

Despite its appearance, this pattern does not introduce true classes. Instead, it is syntactic sugar over the pseudoclassical subclasses pattern, internally relying on prototypes and delegation.

The extends keyword establishes inheritance between two classes by linking the prototype of the subclass to the prototype of the superclass. The super keyword is used to invoke the constructor of the superclass and to access its methods.

1. **Class Declaration:** Defines a constructor and its associated methods in a compact syntax.
2. **Inheritance:** The extends keyword automatically sets up the prototype chain between subclass and superclass.
3. **Super Invocation:** The super(...args) call ensures proper initialization of inherited properties.
4. **Method Sharing:** Methods are automatically placed on the class prototype, ensuring memory efficiency.

This abstraction eliminates the need for manual prototype manipulation (e.g., Object.create) and constructor resetting, reducing the likelihood of errors and improving code readability.

JIT Optimization and Predictability Modern JavaScript engines optimize class-based code similarly to the pseudoclassical pattern. The structured and declarative nature of classes improves predictability, enabling efficient method dispatch and property access.

- **Pros:**
  - Cleaner and more expressive syntax;
  - Built-in support for inheritance via extends;
  - Eliminates manual prototype linking;
  - Reduces common errors (e.g., missing new, incorrect constructor references);

  - Fully compatible with instanceof.

- **Cons:**
  - Still based on prototypes (can obscure underlying behavior);
  - Less flexible than lower-level patterns;
  - Requires understanding of this and super;
  - May give a misleading impression of classical inheritance.

- **Use Cases:**
  - Large-scale applications;
  - Hierarchical models;
  - Codebases requiring readability and maintainability.

10.10.1 Example of Class-Based Pattern usage

Let us rewrite the previous example using the Class-Based Pattern syntax.

```
class Car {
  constructor(position){
    this.position = position;
  }

  move(){
    this.position++;
  }
}

class Van extends Car {
  constructor(position){
    super(position); // equivalent to Car.call(this, position)

    this.grab = function(){
      console.log("I'm grabbing!");
    }
  }
}

// In ES6 classes, the constructor is automatically set correctly
console.log(Van.prototype.constructor); // Van

var vanA = new Van(3);
vanA.move();

var carA = new Car(1);
carA.move();

// Prototype chain check
console.log(vanA.__proto__ === Van.prototype); // true
console.log(vanA.__proto__ === Car.prototype); // false
```

```
console.log(vanA.__proto__ === Car); // false
console.log(Van.prototype === Car.prototype); // false
console.log(Van.prototype.__proto__ === Car.prototype); // true

// instanceof check
console.log(vanA instanceof Van); // true
console.log(vanA instanceof Car); // true
```

At the end of the code we have the prototype chain check confirming that this notation is syntactic sugar over the pseudoclassical subclasses pattern. There is also the instanceof check confirming that the JavaScript engine recognizes that the vanA object belongs to both classes.

10.11 Synthesis and Pattern Comparison Summary

10.11.1 Architectural Comparison Matrix

Pattern	Memory Efficiency	Creation Speed	GC Pressure	Encapsulation	Inheritance Suitability
Decorator	Low	Medium	High	Weak	Low
Functional	Low	Medium	High	Strong	Low
Functional Shared	Medium	Medium	Medium	Weak	Low
Hybrid (Closure + Shared)	Medium	Medium	Medium	Strong	Low
Prototypal	High	Fast	Low	Weak	High
Pseudoclassical	High	Fast	Low	Weak	High
ES6 Class-based	High	Fast	Low	Weak	High

10.11.2 Decision Matrix for Architects

- **Use Pseudoclassical / ES6 Classes:** Default for large-scale applications where memory efficiency, predictable object shapes, and V8 optimization are critical. Suitable for hierarchical domain modeling.
- **Use Prototypal Classes:** When you want explicit control over delegation without relying on class abstractions. Ideal for developers who prefer a more transparent and flexible model closer to JavaScript's core semantics.
- **Use Functional Classes:** When strong encapsulation and private state are non-negotiable and the number of instances is limited.
- **Use Functional Shared Pattern:** When memory optimization is required but encapsulation is not a concern. Suitable for lightweight object factories.
- **Use Hybrid Pattern (Closure + Shared):** When a balance between encapsulation and memory efficiency is required.
- **Use Decorators:** For dynamic runtime extensions, such as plugin systems, middleware, or object augmentation pipelines.

10.11.3 Conceptual Unification

All the patterns presented can be understood as different trade-offs built on top of the same fundamental mechanism: **object composition and prototype delegation**. The primary design dimensions are:

- **Where methods are stored:** inside the object (per-instance) or on a shared prototype.
- **How state is managed:** via closures (private) or object properties (public).
- **How reuse is achieved:** through composition, delegation, or inheritance.

From this perspective, the evolution of patterns can be seen as a progression:

Decorator → Functional → Shared → Prototypal → Pseudoclassical → ES6 Classes

Each step improves either memory efficiency, structure, or developer ergonomics, often at the cost of flexibility or transparency.

10.11.4 Final Remarks

In modern development, the ES6 class syntax is the industry standard. However, it is vital to remember that class is merely **syntactic sugar** over the pseudoclassical pattern. Masking the underlying prototype logic with classes can lead to architectural fragility and context misunderstandings if the developer does not fully grasp the delegation model occurring beneath the abstraction.

In particular:

- extends automatically links prototype chains;
- super replaces manual Superclass.call(this, ...) invocations;
- method definitions are placed on the prototype, not on instances.

Mastering prototypes, delegation, and execution context is essential for reasoning about performance, debugging inheritance issues, and designing scalable architectures. The ability to move fluently between these patterns is a distinguishing characteristic of a Senior JavaScript Engineer.